

HookSuite — Manual Técnico P5 IA + GitHub

Proyecto: HookSuite

Rol: P5 — Módulo de IA + Responsable de GitHub

Stack: Python + Anthropic API + GitHub Actions + LaTeX/Typst

Última actualización: Abril 2026

Antes de empezar

Este manual es tu guía completa de trabajo en el proyecto HookSuite. Está diseñado para que puedas trabajar de forma autónoma desde cualquier lugar y en cualquier momento.

Cuando un paso requiere coordinación con otro rol:

👉 **CONTACTO CON P1 Frontend** — Lo que necesitas comunicarle y cuándo.

Cuando la IA puede ayudarte:

🤖 **PROMPT DE IA** — Copia este prompt en Claude Code o Claude.ai.

Sobre tu rol: P5 es el único rol con dos responsabilidades completamente distintas. Por un lado construyes el módulo técnico más complejo del proyecto: el cerebro de IA que analiza vulnerabilidades y orquesta ataques. Por otro lado eres el guardián del repositorio GitHub y el responsable de que el informe PDF final quede profesional. Necesitas capacidad técnica y capacidad de gestión al mismo tiempo.

Tu trabajo en GitHub empieza el día 1 y nunca termina. Es la responsabilidad más transversal del proyecto.

Herramientas necesarias

- **Python 3.11+** — `python --version`
 - **pip** — `pip --version`
 - **Git** — `git --version`
 - **Claude Code** — Cliente terminal de Anthropic
 - **Visual Studio Code** — O el editor que prefieras
 - **Cuenta en Anthropic Console** — Para obtener la API Key: console.anthropic.com
 - **Typst** — Para maquetar el informe PDF. Instala desde typst.app
-

BLOQUE 01 — GitHub: estructura y gobierno del repositorio

Este bloque empieza el día 1 y sus responsabilidades se mantienen activas durante los 30 días del proyecto.

Paso 1 — Crear y estructurar el repositorio

1.1 Ve a github.com y crea un repositorio nuevo con estas opciones:

- Nombre: `hooksuite`

- Visibilidad: Privada
- Inicializar con README: Sí
- .gitignore: Python
- Licencia: Ninguna

1.2 Entra en el repositorio y ve a Settings → Collaborators → Add people. Añade a P1, P2, P3, P4 y al profesor con sus usuarios de GitHub.

1.3 Clona el repositorio en tu máquina:

```
git clone https://github.com/[TU-USUARIO]/hooksuite.git
cd hooksuite
```

1.4 Crea la estructura de carpetas inicial:

```
mkdir -p frontend backend playwright devtools ia docs/capturas/frontend
docs/capturas/backend docs/capturas/playwright docs/capturas/devtools
docs/capturas/ia infra
touch frontend/.gitkeep backend/.gitkeep playwright/.gitkeep devtools/.gitkeep
ia/.gitkeep
```

1.5 Crea el archivo `.gitignore` en la raíz (reemplaza el generado por GitHub):

```
# Python
venv/
__pycache__/
*.py[cod]
*.egg-info/
.env
.env.local

# Node
node_modules/
dist/
build/
.env.local

# Docker
.docker/

# Credenciales y secretos
*.key
*.pem
secrets/

# Resultados locales
*/results/
*.log
```

```
# IDE
.vscode/
.idea/
*.swp

# OS
.DS_Store
Thumbs.db
```

1.6 Crea el archivo **CONTRIBUTING.md** con la convención de commits:

```
# Guía de contribución – HookSuite

## Convención de commits

Todos los commits deben seguir este formato exacto:
```

tipo(scope): descripción en imperativo presente

```
### Tipos permitidos
- `feat` – Nueva funcionalidad
- `fix` – Corrección de errores
- `docs` – Cambios en documentación
- `refactor` – Reestructuración sin cambio de comportamiento
- `test` – Añadir o modificar tests
- `chore` – Tareas de mantenimiento (actualizar dependencias, etc.)

### Scopes disponibles
`frontend` | `backend` | `playwright` | `devtools` | `ia` | `infra` | `docs`

### Ejemplos correctos
```

feat(frontend): panel de peticiones interceptadas con filtros
fix(backend): corregir timeout en proxy HTTP
docs(ia): documentar prompts de análisis de SQLi
chore(infra): actualizar versión de nginx en docker-compose

```
### Ejemplos incorrectos
```

actualizado el frontend ← sin tipo ni scope
feat: cosas varias ← sin scope, descripción vaga
FEAT(FRONTEND):
Panel ← mayúsculas no permitidas

```

## Política de ramas
- `main` – Protegida. Solo recibe merges mediante Pull Request aprobado por P5
- `feature/frontend` – Rama de P1
- `feature/backend` – Rama de P2
- `feature/playwright` – Rama de P3
- `feature/devtools` – Rama de P4
- `feature/ia` – Rama de P5

## Pull Requests
- Mínimo una revisión aprobada (P5) antes de mergear a main
- Sin conflictos sin resolver
- Pipeline de GitHub Actions en verde

```

1.7 Crea el README principal:

```

# HookSuite

**Herramienta de auditoría de seguridad web con IA**

HookSuite es una plataforma de pentesting web que combina interceptación de
tráfico HTTP,
automatización con Playwright y análisis con inteligencia artificial para detectar
vulnerabilidades de forma automatizada.

## Módulos

| Módulo | Descripción | Responsable |
|-----|-----|-----|
| Frontend | Dashboard React con interfaz de usuario | P1 |
| Backend | API FastAPI con proxy HTTP y WebSockets | P2 |
| Playwright | Automatización de ataques con navegador | P3 |
| DevTools | Captura de tráfico con Chrome DevTools Protocol | P4 |
| IA | Análisis y orquestación con Claude API | P5 |

## Stack tecnológico
- **Frontend:** React + Vite + Tailwind CSS
- **Backend:** Python + FastAPI + mitmproxy + WebSockets
- **Automatización:** Playwright + asyncio
- **Captura de red:** Chrome DevTools Protocol
- **IA:** Anthropic Claude API
- **Infraestructura:** Docker + Hetzner Cloud + Nginx + GitHub Actions

## Instalación y despliegue

Ver [/docs/guia\_despliegue.md](docs/guia_despliegue.md)

## Módulo: Ciberseguridad Avanzada | Curso 2026

```

1.8 Commit inicial:

```
git add .  
git commit -m "chore(infra): estructura inicial del repositorio HookSuite"  
git push origin main
```

Paso 2 — Crear las ramas de trabajo para cada rol

2.1 Crea todas las ramas de módulo:

```
git checkout -b feature/frontend && git push origin feature/frontend  
git checkout main  
git checkout -b feature/backend && git push origin feature/backend  
git checkout main  
git checkout -b feature/playwright && git push origin feature/playwright  
git checkout main  
git checkout -b feature/devtools && git push origin feature/devtools  
git checkout main  
git checkout -b feature/ia && git push origin feature/ia  
git checkout main
```

2.2 Verifica que todas las ramas se crearon en GitHub. Ve a [github.com/\[TU-USUARIO\]/hooksuite](https://github.com/[TU-USUARIO]/hooksuite) y haz clic en el selector de ramas. Debes ver las 5 ramas de módulo más **main**.

2.3 Comunica a cada rol su rama:

♥ **CONTACTO CON TODOS LOS ROLES** — Comunica a P1, P2, P3 y P4 que sus ramas están creadas:

- P1 trabaja en **feature/frontend**
- P2 trabaja en **feature/backend**
- P3 trabaja en **feature/playwright**
- P4 trabaja en **feature/devtools**
- Tú trabajas en **feature/ia**

Comparte también el archivo **CONTRIBUTING.md** para que todos conozcan la convención de commits.

Paso 3 — Configurar la protección de la rama main

3.1 Ve a [github.com/\[TU-USUARIO\]/hooksuite](https://github.com/[TU-USUARIO]/hooksuite) → Settings → Branches.

3.2 Haz clic en "Add branch protection rule".

3.3 En "Branch name pattern" escribe **main**.

3.4 Activa estas opciones:

- ☒ Require a pull request before merging

- ☒ Require approvals: 1
- ☒ Dismiss stale pull request approvals when new commits are pushed

3.5 Haz clic en "Create".

A partir de ahora nadie puede hacer push directo a `main`. Todo pasa por Pull Request y necesita tu aprobación.

Paso 4 — Crear el pipeline de GitHub Actions

4.1 Crea la carpeta y el archivo del workflow:

```
mkdir -p .github/workflows
```

4.2 Crea el archivo `.github/workflows/deploy.yml`:

```
name: Deploy HookSuite

on:
  push:
    branches: [ main ]

jobs:
  build-and-deploy:
    runs-on: ubuntu-latest

    steps:
      - name: Checkout código
        uses: actions/checkout@v4

      - name: Setup Node.js
        uses: actions/setup-node@v4
        with:
          node-version: '20'

      - name: Build Frontend
        working-directory: ./frontend
        run: |
          npm ci
          npm run build
        continue-on-error: true

      - name: Setup Python
        uses: actions/setup-python@v4
        with:
          python-version: '3.11'

      - name: Verificar Backend
        working-directory: ./backend
```

```

run: |
  pip install -r requirements.txt
  python -c "import fastapi; print('FastAPI OK')"
continue-on-error: true

- name: Deploy a Hetzner
  uses: appleboy/ssh-action@master
  with:
    host: ${ secrets.HETZNER_IP }
    username: root
    key: ${ secrets.HETZNER_SSH_KEY }
    script: |
      cd /root/hooksuite
      git pull origin main
      docker compose down
      docker compose up -d --build
      echo "Deploy completado: $(date)"

```

4.3 Añade los secrets de GitHub. Ve a Settings → Secrets and variables → Actions → New repository secret:

- **HETZNER_IP** — La IP del servidor Hetzner (P2 te la proporcionará cuando aprovisione el servidor)
- **HETZNER_SSH_KEY** — La clave SSH privada para conectar al servidor

📌 **CONTACTO CON P2 Backend** — Pide a P2 que te proporcione:

1. La IP del servidor Hetzner cuando lo aprovisione
2. La clave SSH privada del servidor para añadirla al secret **HETZNER_SSH_KEY**

4.4 Commit del pipeline:

```

git add .
git commit -m "chore(infra): GitHub Actions pipeline de despliegue automático"
git push origin main

```

Paso 5 — Establecer la rutina de revisión semanal del repositorio

Cada viernes realiza esta auditoría del repositorio. No debería llevarte más de 15 minutos.

5.1 Verifica commits de todos los miembros:

```

git log --all --oneline --since="7 days ago" --author-date-order

```

Cada miembro debe tener al menos 3 commits en la semana. Si alguien no ha commiteado en más de 3 días hábiles, comunícaselo.

5.2 Verifica que los mensajes siguen la convención:

```
git log --all --oneline --since="7 days ago" | grep -v "[0-9a-f]* feat\|[0-9a-f]* fix\|[0-9a-f]* docs\|[0-9a-f]* refactor\|[0-9a-f]* test\|[0-9a-f]* chore"
```

Si aparecen commits que no siguen la convención, comunícaselo al responsable.

5.3 Verifica el estado de los Pull Requests pendientes en github.com y aprueba o solicita cambios.

5.4 Verifica que el README refleja el estado actual del proyecto. Actualízalo si hace falta.

BLOQUE 02 — Módulo de IA: análisis de vulnerabilidades

Paso 6 — Configurar el entorno Python del módulo de IA

6.1 Crea tu rama y navega a la carpeta:

```
git checkout feature/ia
cd ia
```

6.2 Crea el entorno virtual:

```
python -m venv venv
source venv/bin/activate # Mac/Linux
# venv\Scripts\activate # Windows
```

6.3 Crea `requirements.txt`:

```
anthropic==0.25.0
httpx==0.27.0
python-dotenv==1.0.1
```

6.4 Instala las dependencias:

```
pip install -r requirements.txt
```

6.5 Crea el archivo `.env`:

```
ANTHROPIC_API_KEY=tu_api_key_aqui
BACKEND_URL=http://localhost:8000
```



```
MODEL=claude-sonnet-4-20250514
MAX_TOKENS=1000
```

Para obtener la API Key: ve a console.anthropic.com → API Keys → Create Key.

6.6 Crea la estructura de carpetas:

```
mkdir -p prompts analyzers results
touch prompts/__init__.py analyzers/__init__.py
```

Estructura final:

```
ia/
├── prompts/
│   ├── __init__.py
│   ├── network_analysis.py ← Prompt para analizar paquetes de red
│   ├── intruder_analysis.py ← Prompt para analizar resultados del Intruder
│   ├── console_analysis.py ← Prompt para analizar errores de consola
│   └── fingerprint_analysis.py ← Prompt para priorizar ataques
├── analyzers/
│   ├── __init__.py
│   └── vulnerability_classifier.py ← Clasificador de vulnerabilidades
├── results/          ← Resultados de análisis
├── client.py         ← Cliente de la API de Anthropic
├── orchestrator.py   ← Orquestador del ciclo de ataque
├── main.py           ← Punto de entrada
├── requirements.txt
└── .env
```

Paso 7 — Crear el cliente de la API de Anthropic

7.1 Crea `client.py`:

```
import anthropic
import asyncio
import os
import json
from dotenv import load_dotenv
from typing import Optional

load_dotenv()

MODEL = os.getenv("MODEL", "claude-sonnet-4-20250514")
MAX_TOKENS = int(os.getenv("MAX_TOKENS", 1000))
API_KEY = os.getenv("ANTHROPIC_API_KEY")
```

```

if not API_KEY:
    raise ValueError("ANTHROPIC_API_KEY no está configurada en el archivo .env")

_client = anthropic.Anthropic(api_key=API_KEY)

async def call_claude(
    system_prompt: str,
    user_message: str,
    max_tokens: int = MAX_TOKENS,
    max_retries: int = 3,
) -> Optional[str]:
    for attempt in range(max_retries):
        try:
            loop = asyncio.get_event_loop()
            response = await loop.run_in_executor(
                None,
                lambda: _client.messages.create(
                    model=MODEL,
                    max_tokens=max_tokens,
                    system=system_prompt,
                    messages=[{"role": "user", "content": user_message}],
                )
            )
            return response.content[0].text

        except anthropic.RateLimitError:
            wait = 2 ** attempt
            print(f" ⚠ Rate limit. Esperando {wait}s... (intento {attempt + 1}/{max_retries})")
            await asyncio.sleep(wait)

        except anthropic.APIError as e:
            print(f" X Error API de Anthropic: {e}")
            if attempt < max_retries - 1:
                await asyncio.sleep(2)
            else:
                return None

        except Exception as e:
            print(f" X Error inesperado: {e}")
            return None

    return None

async def call_claude_json(
    system_prompt: str,
    user_message: str,
    max_tokens: int = MAX_TOKENS,
) -> Optional[dict]:
    response_text = await call_claude(system_prompt, user_message, max_tokens)
    if not response_text:
        return None
    try:
        clean = response_text.strip()

```

```

        if clean.startswith("`"):
            lines = clean.split('\n')
            clean = '\n'.join(lines[1:-1])
        return json.loads(clean)
    except json.JSONDecodeError as e:
        print(f" X Error parseando JSON de Claude: {e}")
        print(f" Respuesta recibida: {response_text[:200]}")
        return None

```

7.2 Verifica que la API funciona creando `test_api.py` temporal:

```

import asyncio
from client import call_claude

async def main():
    print("Probando conexión con la API de Anthropic...")
    response = await call_claude(
        system_prompt="Eres un asistente de ciberseguridad. Responde en español.",
        user_message="¿Qué es un SQL Injection? Explícalo en una frase.",
        max_tokens=100,
    )
    if response:
        print(f"✓ API funcionando correctamente.")
        print(f" Respuesta: {response}")
    else:
        print("X No se pudo conectar con la API. Verifica tu API Key.")

asyncio.run(main())

```

```
python test_api.py
```

Debes ver la API respondiendo. Guarda la salida y elimina el script:

```
python test_api.py > results/test_api_output.txt
rm test_api.py
```

 **PROMPT DE IA** — Si tienes problemas con la API Key o la conexión:

Estoy intentando conectar con la API de Anthropic en Python usando la librería oficial `anthropic` para el proyecto HookSuite. Recibo este error: [pega el error exacto]. Mi código de conexión es: [pega el código]. ¿Cómo lo soluciono?

Paso 8 — Crear el prompt de análisis de paquetes de red

Este prompt analiza cada paquete capturado por P4 y determina si hay indicios de vulnerabilidad.

8.1 Crea `prompts/network_analysis.py`:

```
SYSTEM_PROMPT = """Eres un experto en seguridad web especializado en análisis de
tráfico HTTP.
Recibes un paquete de red capturado durante una auditoría de seguridad y debes
analizarlo
buscando indicios de vulnerabilidades OWASP Top 10.

REGLAS IMPORTANTES:
- Responde ÚNICAMENTE con JSON válido, sin texto adicional ni marcadores de código
- Sé conservador: solo marca como vulnerable si hay evidencia clara
- El campo "confianza" debe ser 0-100 basado en la solidez de la evidencia

FORMATO DE RESPUESTA OBLIGATORIO:
{
  "vulnerable": true/false,
  "tipo": "SQL Injection" | "XSS" | "IDOR" | "Path Traversal" | "Command
Injection" | "None",
  "severidad": "critical" | "high" | "medium" | "low" | "none",
  "confianza": 0-100,
  "justificacion": "Explicación clara en español de por qué es o no es
vulnerable",
  "recomendacion": "Paso concreto para remediar la vulnerabilidad (vacío si no es
vulnerable)"
}"""

def build_user_message(packet: dict) -> str:
    url = packet.get("url", "")
    method = packet.get("method", "GET")
    status = packet.get("status", 0)
    time_ms = packet.get("time", 0)
    request_headers = packet.get("request_headers", {})
    request_body = packet.get("request_body") or "(sin body)"
    response_headers = packet.get("response_headers", {})
    response_body = (packet.get("response_body") or "")[:2000]
    suspicious_reasons = packet.get("suspicious_reasons", [])

    return f"""Analiza este paquete HTTP capturado durante una auditoría de
HookSuite:

PETICIÓN:
- Método: {method}
- URL: {url}
- Headers relevantes: {dict(list(request_headers.items())[:5])}
- Body: {request_body[:500]}

RESPUESTA:
- Status: {status}
- Tiempo de respuesta: {time_ms}ms
```

```
- Headers: {dict(list(response_headers.items())[:5])}
- Body (primeros 2000 chars): {response_body}

SEÑALES DETECTADAS PREVIAMENTE:
{suspicious_reasons if suspicious_reasons else "Ninguna señal previa detectada"}

Determina si este paquete evidencia alguna vulnerabilidad de seguridad."""
```

8.2 Crea prompts/intruder_analysis.py:

```
import json

SYSTEM_PROMPT = """Eres un experto en análisis de resultados de pruebas de
penetración web.
Recibes los resultados de un ataque de fuzzing (Intruder) y debes identificar qué
payloads
provocaron comportamientos anómalos que sugieren vulnerabilidades.

REGLAS:
- Responde ÚNICAMENTE con JSON válido
- Busca anomalías estadísticas: payloads que generan respuestas significativamente
distintas
- Considera: diferencias de tamaño, tiempos de respuesta, status codes, patrones
en el body

FORMATO DE RESPUESTA:
{
  "anomalous_payloads": [
    {
      "payload": "el payload exacto",
      "reason": "por qué es anómalo",
      "probable_vulnerability": "tipo de vulnerabilidad probable",
      "confidence": 0-100
    }
  ],
  "summary": "Resumen del análisis en una frase",
  "recommended_followup": "Qué hacer a continuación para confirmar"
}"""

def build_user_message(results: list, attack_type: str) -> str:
    if not results:
        return "No hay resultados que analizar."

    normal_results = [r for r in results if r.get("status", 0) == 200]
    normal_size = sum(r.get("size", 0) for r in normal_results) //
max(len(normal_results), 1)
    normal_time = sum(r.get("time", 0) for r in normal_results) //
max(len(normal_results), 1)

    results_summary = []
    for r in results[:30]:
        results_summary.append({
```

```

        "payload": r.get("payload", ""),
        "status": r.get("status", 0),
        "size": r.get("size", 0),
        "time": r.get("time", 0),
        "size_diff_from_normal": r.get("size", 0) - normal_size,
        "time_diff_from_normal": r.get("time", 0) - normal_time,
    })

```

return f"""Analiza estos resultados de ataque tipo '{attack_type}' del Intruder de HookSuite:

ESTADÍSTICAS DE REFERENCIA:

- Tamaño medio de respuesta normal: {normal_size} bytes
- Tiempo medio de respuesta normal: {normal_time}ms
- Total de payloads probados: {len(results)}

RESULTADOS (máximo 30):

```
{json.dumps(results_summary, indent=2, ensure_ascii=False)}
```

Identifica qué payloads provocaron comportamientos anómalos."""

8.3 Crea prompts/console_analysis.py:

```
import json
```

SYSTEM_PROMPT = """Eres un experto en análisis de seguridad de logs de aplicaciones web.
Recibes mensajes de la consola del navegador capturados durante una auditoría y debes
identificar qué información sensible está siendo expuesta.

REGLAS:

- Responde ÚNICAMENTE con JSON válido
- Clasifica cada hallazgo por tipo y severidad real
- Un stack trace es medio, una password expuesta es crítico

FORMATO DE RESPUESTA:

```

{
  "findings": [
    {
      "message_snippet": "primeros 100 chars del mensaje",
      "tipo": "SERVER_PATH" | "API_KEY" | "PASSWORD" | "SQL_ERROR" | "STACK_TRACE"
| "INTERNAL_IP" | "OTHER",
      "severidad": "critical" | "high" | "medium" | "low",
      "riesgo": "Explicación del riesgo de seguridad en español",
      "recomendacion": "Cómo remediar esto"
    }
  ],
  "summary": "Resumen de los hallazgos más importantes"
}"""

```

```
def build_user_message(console_findings: list) -> str:
```

```

if not console_findings:
    return "No hay hallazgos de consola que analizar."

formatted = json.dumps([
    {
        "level": f.get("level"),
        "text": f.get("text", "")[:300],
        "detections_previas": f.get("detections", []),
    }
    for f in console_findings[:20]
], indent=2, ensure_ascii=False)

return f"""Analiza estos mensajes de consola del navegador capturados por
HookSuite:

MENSAJES DE CONSOLA ({len(console_findings)} hallazgos previos):
{formatted}

Determina el riesgo de seguridad real de cada mensaje y qué información sensible
expone."""

```

8.4 Crea `prompts/fingerprint_analysis.py`:

```

import json

SYSTEM_PROMPT = """Eres un experto en threat modeling y pentesting web.
Recibes el perfil tecnológico de una aplicación web (stack detectado) y debes
determinar qué vectores de ataque son más probables y en qué orden atacarlos.

REGLAS:
- Responde ÚNICAMENTE con JSON válido
- Basa las prioridades en vulnerabilidades conocidas del stack detectado
- Sé específico: no digas 'SQLi', di 'SQLi en campos de búsqueda PHP/MySQL'

FORMATO DE RESPUESTA:
{
  "attack_priorities": [
    {
      "tipo": "nombre del ataque",
      "razon": "por qué este stack es vulnerable a este ataque",
      "prioridad": 1-10,
      "parametros_objetivo": ["campo1", "campo2"]
    }
  ],
  "summary": "Resumen del perfil de amenaza en una frase"
}"""

def build_user_message(fingerprint: dict) -> str:
    return f"""Analiza el perfil tecnológico de este objetivo detectado por
HookSuite:

TECNOLOGÍAS DETECTADAS: {fingerprint.get('technologies', [])}

```

```
SERVIDOR: {fingerprint.get('server', 'Desconocido')}
POWERED BY: {fingerprint.get('powered_by', 'Desconocido')}
URL: {fingerprint.get('url', '')}
```

Determina los vectores de ataque más probables y en qué orden atacarlos."""

Paso 9 — Crear el clasificador de vulnerabilidades

9.1 Crea `analyzers/vulnerability_classifier.py`:

```
import uuid
from datetime import datetime
from typing import Dict, Optional
import json
import os

from client import call_claude_json
from prompts.network_analysis import SYSTEM_PROMPT as NET_SYSTEM,
build_user_message as net_msg
from prompts.intruder_analysis import SYSTEM_PROMPT as INT_SYSTEM,
build_user_message as int_msg
from prompts.console_analysis import SYSTEM_PROMPT as CON_SYSTEM,
build_user_message as con_msg
from prompts.fingerprint_analysis import SYSTEM_PROMPT as FIN_SYSTEM,
build_user_message as fin_msg

os.makedirs("results", exist_ok=True)

class VulnerabilityClassifier:
    def __init__(self):
        self.analyses_count = 0
        self.vulnerabilities_found = []

    async def analyze_packet(self, packet: dict) -> Optional[dict]:
        self.analyses_count += 1
        print(f" 🔍 Analizando paquete #{self.analyses_count}:
{packet.get('url', '')[:60]}")

        result = await call_claude_json(NET_SYSTEM, net_msg(packet))

        if not result:
            return None

        if result.get("vulnerable") and result.get("confianza", 0) >= 60:
            vulnerability = self._build_vulnerability(packet, result,
"network_packet")
            self.vulnerabilities_found.append(vulnerability)
            print(f" ⚠️ VULNERABILIDAD: {vulnerability['tipo']}
({vulnerability['severidad']}) – confianza: {result['confianza']}%")
            return vulnerability
```



```
        return None

    async def analyze_intruder_results(self, results: list, attack_type: str) -> Optional[dict]:
        if not results:
            return None

        print(f" 🔍 Analizando {len(results)} resultados del Intruder ({attack_type})...")
        analysis = await call_claude_json(INT_SYSTEM, int_msg(results, attack_type))

        if not analysis:
            return None

        anomalous = analysis.get("anomalous_payloads", [])
        high_confidence = [a for a in anomalous if a.get("confidence", 0) >= 70]

        if high_confidence:
            print(f" ⚠️ {len(high_confidence)} payload(s) anómalo(s) detectados por IA")
            return {
                "type": "intruder_analysis",
                "anomalous_payloads": high_confidence,
                "summary": analysis.get("summary", ""),
                "recommended_followup": analysis.get("recommended_followup", ""),
            }

        return None

    async def analyze_console_findings(self, findings: list) -> Optional[dict]:
        if not findings:
            return None

        print(f" 🔍 Analizando {len(findings)} hallazgos de consola...")
        analysis = await call_claude_json(CON_SYSTEM, con_msg(findings))

        if not analysis:
            return None

        critical_findings = [f for f in analysis.get("findings", [])
                             if f.get("severidad") in ["critical", "high"]]
        if critical_findings:
            print(f" ⚠️ {len(critical_findings)} hallazgo(s) críticos en consola")

        return analysis

    async def analyze_fingerprint(self, fingerprint: dict) -> Optional[dict]:
        print(f" 🔍 Analizando fingerprint: {fingerprint.get('technologies', [])}")
        analysis = await call_claude_json(FIN_SYSTEM, fin_msg(fingerprint))
        if analysis:
```

```

        priorities = analysis.get("attack_priorities", [])
        print(f"  ✓ Prioridades de ataque: {[p['tipo'] for p in
priorities[:3]]}")
        return analysis

    def _build_vulnerability(self, source: dict, analysis: dict, source_type: str)
-> dict:
        return {
            "id": str(uuid.uuid4()),
            "tipo": analysis.get("tipo", "Unknown"),
            "severidad": analysis.get("severidad", "medium"),
            "titulo": f"{analysis.get('tipo', 'Vulnerabilidad')} detectada en
{source.get('url', 'URL desconocida')[:60]}",
            "descripcion": analysis.get("justificacion", ""),
            "url": source.get("url", ""),
            "payload": source.get("request_body") or source.get("url", ""),
            "recomendacion": analysis.get("recomendacion", ""),
            "confianza": analysis.get("confianza", 0),
            "source_type": source_type,
            "timestamp": datetime.utcnow().isoformat(),
        }

    def get_vulnerabilities(self) -> list:
        return self.vulnerabilities_found.copy()

    def save_results(self, filepath: str = "results/vulnerabilities.json"):
        with open(filepath, 'w') as f:
            json.dump({
                "total_analyses": self.analyses_count,
                "vulnerabilities_found": len(self.vulnerabilities_found),
                "vulnerabilities": self.vulnerabilities_found,
            }, f, indent=2, ensure_ascii=False)
            print(f"✓ {len(self.vulnerabilities_found)} vulnerabilidades guardadas en
{filepath}")

```

Paso 10 — Commit del bloque 02

```

git add .
git commit -m "feat(ia): cliente Anthropic, prompts de análisis y clasificador de
vulnerabilidades"
git push origin feature/ia

```

BLOQUE 03 — Módulo de IA: orquestación de ataques

Paso 11 — Crear el orquestador del ciclo de ataque

📖 **CONTACTO CON P2 Backend** — Para que el orquestador funcione necesitas que P2 tenga listo el servidor. La URL que usarás para enviar instrucciones a Playwright es `POST http://localhost:8000/api/playwright/instruction/{session_token}`. Cuando P2 confirme que ese endpoint está disponible, activa el envío.

📖 **CONTACTO CON P3 Playwright** — El orquestador le enviará instrucciones a P3 en este formato JSON. Confirma con P3 que este es el formato que espera recibir:

```
{
  "type": "attack",
  "url": "http://objetivo.com/pagina",
  "selector": "input[name='id']",
  "payload": "' OR '1'='1'",
  "verify": "admin",
  "session_token": "uuid-de-la-sesion"
}
```

11.1 Crea `orchestrator.py`:

```
import asyncio
import httpx
import json
import os
from typing import Dict, List, Optional
from dotenv import load_dotenv
from datetime import datetime

from analyzers.vulnerability_classifier import VulnerabilityClassifier

load_dotenv()
BACKEND_URL = os.getenv("BACKEND_URL", "http://localhost:8000")

SQLI_PAYLOADS_PRIORITY = [
    "' OR '1'='1'",
    "' OR '1'='1'--",
    "1 UNION SELECT null--",
    "1 UNION SELECT null,null--",
    "' AND SLEEP(5)--",
]

XSS_PAYLOADS_PRIORITY = [
    "<script>alert(1)</script>",
    "<img src=x onerror=alert(1)>",
    "'><script>alert(1)</script>",
]

class AttackOrchestrator:
    def __init__(self, session_token: str):
        self.session_token = session_token
        self.classifier = VulnerabilityClassifier()
```

```

        self.backend_available = False
        self.confirmed_vulnerabilities = []

    async def check_backend(self) -> bool:
        try:
            async with httpx.AsyncClient(timeout=5) as client:
                response = await client.get(f"{BACKEND_URL}/health")
                self.backend_available = response.status_code == 200
                return self.backend_available
        except Exception:
            self.backend_available = False
            return False

    async def send_instruction_to_playwright(self, instruction: dict) -> Optional[dict]:
        if not self.backend_available:
            print(f" ⚠️ Backend no disponible. Instrucción no enviada: {instruction.get('type')}")
            return None

        try:
            async with httpx.AsyncClient(timeout=30) as client:
                response = await client.post(
                    f"{BACKEND_URL}/api/playwright/instruction/{self.session_token}",
                    json={**instruction, "session_token": self.session_token}
                )
                if response.status_code == 200:
                    return response.json()
        except Exception as e:
            print(f" ✖️ Error enviando instrucción a Playwright: {e}")
            return None

    async def send_vulnerability_to_backend(self, vulnerability: dict) -> bool:
        if not self.backend_available:
            return False
        try:
            async with httpx.AsyncClient(timeout=10) as client:
                response = await client.post(
                    f"{BACKEND_URL}/api/vulnerabilities",
                    json=vulnerability
                )
                return response.status_code == 200
        except Exception:
            return False

    async def run_fingerprint_phase(self, target_url: str) -> Optional[dict]:
        print(f"\n📄 FASE 1: Fingerprinting de {target_url}")
        instruction = {"type": "fingerprint", "url": target_url}
        result = await self.send_instruction_to_playwright(instruction)

        if result and result.get("result"):
            fingerprint = result["result"]
            analysis = await self.classifier.analyze_fingerprint(fingerprint)

```

```

        return analysis
    return None

    async def run_spider_phase(self, target_url: str, max_pages: int = 10) ->
List[str]:
        print(f"\n📄 FASE 2: Spider de {target_url}")
        instruction = {"type": "spider", "url": target_url, "max_pages":
max_pages}
        result = await self.send_instruction_to_playwright(instruction)

        if result:
            return result.get("discovered_urls", [])
        return []

    async def run_attack_phase(
        self,
        target_url: str,
        field_selector: str,
        attack_priorities: List[str],
    ) -> List[dict]:
        print(f"\n📄 FASE 3: Ataques en {target_url}")
        found_vulnerabilities = []

        for attack_type in attack_priorities[:3]:
            if attack_type in ["sqli", "blind_sqli"]:
                payloads = SQLI_PAYLOADS_PRIORITY
            elif attack_type == "xss":
                payloads = XSS_PAYLOADS_PRIORITY
            else:
                payloads = ['"', "'", '<script>', '..../']

            for payload in payloads[:5]:
                print(f" → Probando {attack_type}: {payload[:40]}")

                instruction = {
                    "type": "attack",
                    "url": target_url,
                    "selector": field_selector,
                    "payload": payload,
                    "verify": "admin" if "sqli" in attack_type else "alert",
                }

                result = await self.send_instruction_to_playwright(instruction)

                if result and result.get("response_body_snippet"):
                    fake_packet = {
                        "url": target_url,
                        "method": "GET",
                        "status": 200,
                        "time": result.get("time_ms", 0),
                        "request_body": payload,
                        "response_body": result.get("response_body_snippet", ""),
                        "suspicious_reasons": result.get("anomalies", []),
                    }

```

```

        vulnerability = await
self.classifier.analyze_packet(fake_packet)

        if vulnerability:
            confirmed = await self.confirm_vulnerability(
                target_url, field_selector, payload, vulnerability
            )
            if confirmed:
                found_vulnerabilities.append(vulnerability)
                await
self.send_vulnerability_to_backend(vulnerability)
                break

            await asyncio.sleep(0.5)

        return found_vulnerabilities

    async def confirm_vulnerability(
        self,
        url: str,
        selector: str,
        payload: str,
        initial_finding: dict,
        confirmations_needed: int = 2,
    ) -> bool:
        print(f" 📡 Confirmando vulnerabilidad ({confirmations_needed}
confirmaciones)...")
        confirmed_count = 0

        for i in range(confirmations_needed):
            instruction = {
                "type": "attack",
                "url": url,
                "selector": selector,
                "payload": payload,
                "verify": "",
            }
            result = await self.send_instruction_to_playwright(instruction)

            if result and result.get("vulnerable"):
                confirmed_count += 1

            if confirmed_count >= confirmations_needed:
                print(f" ✓ Vulnerabilidad CONFIRMADA
({confirmed_count}/{confirmations_needed})")
                self.confirmed_vulnerabilities.append(initial_finding)
                return True

            print(f" → Vulnerabilidad no confirmada
({confirmed_count}/{confirmations_needed})")
            return False

    async def run_full_audit(self, target_url: str, field_selector: str =

```

```



```

Paso 12 — Crear el punto de entrada del módulo IA

12.1 Crea `main.py`:

```

import asyncio
import os
from dotenv import load_dotenv
from orchestrator import AttackOrchestrator

load_dotenv()

```

```
TARGET_URL = "http://localhost:8888"
FIELD_SELECTOR = "input[name='id']"
SESSION_TOKEN = "ia_session_default"

async def main():
    orchestrator = AttackOrchestrator(session_token=SESSION_TOKEN)
    await orchestrator.run_full_audit(
        target_url=TARGET_URL,
        field_selector=FIELD_SELECTOR,
    )

if __name__ == "__main__":
    asyncio.run(main())
```

12.2 Ejecuta el orquestador en modo standalone (sin Playwright conectado, para verificar que los prompts y el clasificador funcionan):

```
import asyncio
from analyzers.vulnerability_classifier import VulnerabilityClassifier

async def test_standalone():
    classifier = VulnerabilityClassifier()

    print("Test 1: Análisis de paquete con SQLi...")
    test_packet = {
        "url": "http://localhost:8888/vulnerabilities/sqli/?id=' OR '1'='1",
        "method": "GET",
        "status": 200,
        "time": 145,
        "request_body": None,
        "response_body": "ID: admin First name: admin Surname: admin ID: user\nFirst name: Gordon Surname: Brown",
        "suspicious_reasons": ["INJECTION_CHAR_IN_REQUEST"],
    }
    result = await classifier.analyze_packet(test_packet)
    print(f" Resultado: {result}")

    print("\nTest 2: Análisis de fingerprint...")
    test_fingerprint = {
        "url": "http://localhost:8888",
        "technologies": ["PHP", "Apache", "MySQL", "jQuery"],
        "server": "Apache/2.4.51",
        "powered_by": "PHP/7.4.3",
    }
    analysis = await classifier.analyze_fingerprint(test_fingerprint)
    print(f" Prioridades: {[p['tipo'] for p in analysis.get('attack_priorities', [])][:3]}")

    classifier.save_results("results/test_standalone.json")
    print("\n✓ Tests standalone completados")
```



```
asyncio.run(test_standalone())
```

Guarda este script como `test_standalone.py`, ejecútalo y guarda la salida:

```
python test_standalone.py > results/test_standalone_output.txt
```

Elimina el script cuando verifiques que funciona:

```
rm test_standalone.py
```

12.3 Commit del bloque 03:

```
git add .  
git commit -m "feat(ia): orquestador de ataques y punto de entrada del módulo IA"  
git push origin feature/ia
```

BLOQUE 04 — Informe PDF: cohesión y maquetación

Paso 13 — Establecer la fecha límite interna de documentación

👉 **CONTACTO CON TODOS LOS ROLES** — Comunica a P1, P2, P3 y P4 que tienen hasta el **día 20 del proyecto** para enviarte su sección de documentación. Cada uno debe enviarte:

- P1: documentación del frontend y manual de uso (en Markdown)
- P2: arquitectura técnica y guía de despliegue (en Markdown)
- P3: documentación de Playwright con evidencias (en Markdown)
- P4: documentación de DevTools y road map (en Markdown)

Crea un issue en GitHub para recordárselo: Issues → New Issue → "Documentación para informe PDF — Fecha límite: día 20"

Paso 14 — Instalar Typst y preparar la plantilla del informe

14.1 Instala Typst desde typst.app o con el gestor de paquetes:

En Mac:

```
brew install typst
```

En Windows (con winget):

```
winget install --id Typst.Typst
```

En Linux:

```
curl -fsSL https://typst.app/install.sh | sh
```

Verifica la instalación:

```
typst --version
```

14.2 Crea la carpeta del informe:

```
mkdir -p docs/informe  
cd docs/informe
```

14.3 Crea el archivo principal del informe `docs/informe/main.typ`:

```
#import "template.typ": *  
  
#show: project.with(  
  title: "HookSuite",  
  subtitle: "Herramienta de auditoría de seguridad web con Inteligencia  
Artificial",  
  authors: (  
    (name: "P1 – Frontend", role: "Módulo de interfaz de usuario"),  
    (name: "P2 – Backend", role: "Módulo de servidor y proxy HTTP"),  
    (name: "P3 – Playwright", role: "Módulo de automatización"),  
    (name: "P4 – DevTools", role: "Módulo de captura de red"),  
    (name: "P5 – IA + GitHub", role: "Módulo de inteligencia artificial"),  
  ),  
  date: "Mayo 2026",  
  module: "Ciberseguridad Avanzada | Curso 2026",  
)  
  
// =====  
// 1. RESUMEN EJECUTIVO  
// =====  
  
= Resumen ejecutivo  
  
HookSuite es una plataforma web de auditoría de seguridad que combina  
interceptación de
```

tráfico HTTP en tiempo real, automatización de ataques con Playwright y análisis con inteligencia artificial para detectar vulnerabilidades de forma automatizada.

La herramienta implementa las funcionalidades core de Burp Suite (proxy interceptor, repeater, intruder) en una arquitectura cliente-servidor accesible desde el navegador, sin necesidad de instalar software adicional más allá de configurar el archivo PAC de proxy.

Durante las pruebas realizadas sobre DVWA (Damn Vulnerable Web Application), HookSuite detectó con éxito vulnerabilidades de tipo SQL Injection y Blind SQL Injection con una tasa de confirmación superior al 85%.

```
// =====
// 2. DESCRIPCIÓN DEL PROBLEMA
// =====
```

= Descripción del problema y justificación

Las herramientas de pentesting web profesionales como Burp Suite requieren instalación local, licencias de pago y conocimiento técnico avanzado para su configuración. Esto limita su accesibilidad para equipos de seguridad distribuidos o en formación.

HookSuite resuelve este problema ofreciendo una alternativa web-based con las funcionalidades esenciales de auditoría, potenciada con inteligencia artificial para automatizar el análisis de vulnerabilidades.

```
// =====
// 3. ARQUITECTURA TÉCNICA
// =====
```

= Arquitectura técnica

```
#figure(
  image("../capturas/arquitectura.png", width: 100%),
  caption: "Diagrama de arquitectura técnica de HookSuite"
)
```

== Stack tecnológico

```
*Frontend:* React 18 + Vite + Tailwind CSS \
*Backend:* Python 3.11 + FastAPI + mitmproxy + WebSockets \
*Automatización:* Playwright 1.44 + asyncio \
*Captura de red:* Chrome DevTools Protocol (CDP) \
*Inteligencia Artificial:* Anthropic Claude API (claude-sonnet-4-20250514) \
*Infraestructura:* Docker + Hetzner Cloud CX22 + Nginx + GitHub Actions
```

```
// =====
// SECCIONES DE CADA MÓDULO (añadir contenido de cada rol)
// =====

= Módulo Frontend (P1)

// Insertar contenido de P1 aquí

= Módulo Backend (P2)

// Insertar contenido de P2 aquí

= Módulo Playwright (P3)

// Insertar contenido de P3 aquí

= Módulo Chrome DevTools (P4)

// Insertar contenido de P4 aquí

= Módulo de Inteligencia Artificial (P5)

// Insertar contenido de P5 aquí

// =====
// GUÍA DE DESPLIEGUE
// =====

= Guía de despliegue

// Insertar guía de despliegue de P2 aquí

// =====
// MANUAL DE USO
// =====

= Manual de uso

// Insertar manual de uso de P1 aquí

// =====
// CONCLUSIONES
// =====

= Conclusiones y lecciones aprendidas

== Conclusiones técnicas

HookSuite demuestra que es posible construir una herramienta de pentesting web funcional y accesible combinando tecnologías modernas de automatización de navegador con modelos de lenguaje avanzados. La integración del Chrome DevTools Protocol con el análisis de IA
```

proporciona una capacidad de detección superior a las herramientas basadas únicamente en reglas estáticas.

== Lecciones aprendidas

// Completar con reflexiones reales del equipo

// =====
// ROAD MAP
// =====

= Road map – Práctica 2

// Insertar road map de P4 aquí

14.4 Crea la plantilla `docs/informe/template.typ`:

```
#let project(
  title: "",
  subtitle: "",
  authors: (),
  date: "",
  module: "",
  body,
) = {
  set document(author: authors.map(a => a.name), title: title)
  set page(
    margin: (left: 3cm, right: 2.5cm, top: 2.5cm, bottom: 2.5cm),
    numbering: "1",
    number-align: center,
  )
  set text(font: "New Computer Modern", lang: "es", size: 11pt)
  set heading(numbering: "1.1")
  set par(justify: true, leading: 0.8em)

  // Portada
  page(numbering: none)[
    #align(center)[
      #v(2cm)
      #text(size: 14pt, weight: "bold")[#module]
      #v(1cm)
      #line(length: 100%, stroke: 2pt)
      #v(0.5cm)
      #text(size: 28pt, weight: "bold")[#title]
      #v(0.3cm)
      #text(size: 16pt)[#subtitle]
      #v(0.5cm)
      #line(length: 100%, stroke: 2pt)
      #v(2cm)

      #for author in authors [
```

```

    #grid(
      columns: (1fr, 2fr),
      gutter: 1em,
      align(right)[#text(weight: "bold")[#author.name]],
      align(left)[#author.role],
    )
    #v(0.3cm)
  ]

  #v(2cm)
  #text(size: 14pt)[#date]
]
]

// Índice
page(numbering: none)[
  #outline(depth: 2, indent: true)
]

// Cuerpo
body
}

```

14.5 Verifica que Typst puede compilar el informe:

```

cd docs/informe
typst compile main.typ informe_hooksuite.pdf

```

Si no hay errores, se crea el archivo `informe_hooksuite.pdf`. Ábrelo para verificar que la portada y el índice se generan correctamente.



PROMPT DE IA — Para convertir el contenido Markdown de cada rol a formato Typst:

Tengo este contenido en formato Markdown que forma parte del informe técnico del proyecto HookSuite: [pega el contenido Markdown]. Conviértelo a sintaxis Typst manteniendo los títulos, listas, bloques de código y énfasis. El resultado debe ser compatible con Typst 0.11.

Paso 15 — Redactar las secciones transversales del informe

Estas secciones son tuyas exclusivamente. Nadie más puede escribirlas porque requieren visión de todo el proyecto.

15.1 Completa el resumen ejecutivo en `main.typ`. Debe tener como máximo una página y responder: qué hace HookSuite, por qué es relevante, qué tecnologías usa y qué resultados se obtuvieron en las pruebas.

15.2 Completa la sección de descripción del problema. Contextualiza por qué existe HookSuite en el ecosistema de herramientas de seguridad.

15.3 Escribe las conclusiones y lecciones aprendidas. Deben ser reflexivas y concretas, no genéricas. Incluye:

- Qué funcionó mejor de lo esperado
- Qué fue más difícil de lo previsto
- Qué haría el equipo diferente si empezara de nuevo
- Qué aprendió cada módulo sobre integración de sistemas complejos

 **PROMPT DE IA** — Para redactar las conclusiones:

```
Estoy redactando las conclusiones del informe técnico del proyecto HookSuite, una herramienta de pentesting web con IA construida en 30 días por 5 personas. El proyecto incluye: proxy HTTP con mitmproxy, automatización con Playwright, captura de tráfico con Chrome DevTools Protocol y análisis con la API de Claude. Las principales dificultades técnicas fueron: [describe las reales]. Redacta unas conclusiones técnicas de 300-400 palabras en español, reflexivas y concretas, en formato Typst.
```

Paso 16 — Integrar las secciones de cada rol

Cuando recibas la documentación de cada rol (antes del día 20), intégrala en `main.typ`.

16.1 Para cada sección recibida en Markdown, conviértela a Typst con el prompt de IA del paso 14.

16.2 Añade las capturas de pantalla en los lugares correspondientes. Las capturas están en `/docs/capturas/[modulo]/`. En Typst:

```
#figure(  
  image("../capturas/frontend/proxy_panel_peticiones.png", width: 90%),  
  caption: "Panel de peticiones interceptadas del módulo Proxy"  
)
```

16.3 Verifica que el informe compila sin errores después de cada integración:

```
cd docs/informe  
typst compile main.typ informe_hooksuite.pdf
```

16.4 Verifica que el informe contiene los 10 puntos obligatorios del enunciado:

- ☐ Portada con nombre, integrantes y fecha
- ☐ Índice de contenidos
- ☐ Resumen ejecutivo (máx. 1 página)
- ☐ Descripción del problema y justificación

- ☐ Arquitectura técnica con diagrama
 - ☐ Proceso de desarrollo con evidencias fase a fase
 - ☐ Guía de despliegue paso a paso
 - ☐ Manual de uso de la herramienta
 - ☐ Conclusiones y lecciones aprendidas
 - ☐ Road map de mejora para la Práctica 2
-

Paso 17 — Generar el PDF final

17.1 Realiza la compilación final del informe:

```
cd docs/informe
typst compile main.typ informe_hooksuite.pdf
```

17.2 Abre el PDF y verifica visualmente:

- Portada con el diseño correcto
- Índice con números de página correctos
- Imágenes nítidas y con captions
- Código con syntax highlighting
- Numeración de páginas correcta
- Sin errores tipográficos visibles

17.3 Commit del informe:

```
git add .
git commit -m "docs: informe PDF final HookSuite listo para entrega"
git push origin feature/ia
```

BLOQUE 05 — Gestión de Pull Requests y entrega final

Paso 18 — Revisar y mergear los Pull Requests

A medida que los roles vayan terminando su trabajo y abriendo Pull Requests, tú los revisas y mergeas a main.

Para cada Pull Request recibido:

18.1 Lee el código de los cambios en GitHub.

18.2 Verifica que los commits siguen la convención definida en [CONTRIBUTING.md](#).

18.3 Verifica que no hay credenciales, API keys ni archivos `.env` subidos al repositorio:


```
git log --all --full-history -- "**/.env" "**/*.key" "**/secrets*"
```

Si aparece algún resultado, el commit incluye credenciales. Pide al rol correspondiente que lo limpie con `git filter-branch` o `BFG Repo Cleaner`.

18.4 Aprueba el PR y haz el merge a main.

18.5 Verifica que el pipeline de GitHub Actions se ejecuta correctamente después del merge. Ve a `github.com/[USUARIO]/hooksuite` → Actions.

 **PROMPT DE IA** — Si hay conflictos de merge que no sabes resolver:

Tengo un conflicto de merge en Git en el proyecto HookSuite. El conflicto está en el archivo [nombre del archivo]. El contenido del conflicto es: [pega el contenido con los marcadores <<<, === y >>>]. ¿Cómo lo resuelvo correctamente preservando los cambios de ambas ramas?

Paso 19 — Verificar el estado final del repositorio

Antes del día 22, realiza la auditoría final del repositorio.

19.1 Verifica que todos los módulos están en main:

```
git log --oneline --graph main | head -30
```

19.2 Verifica que el historial de commits es limpio y descriptivo:

```
git log --oneline main | head -50
```

19.3 Verifica la estructura final del repositorio:

```
find . -type f -name "*.py" -o -name "*.jsx" -o -name "*.js" | grep -v node_modules | grep -v venv | grep -v __pycache__ | sort
```

19.4 Actualiza el README con el estado final del proyecto. Añade:

- URL de la herramienta desplegada en Hetzner
- Screenshots del dashboard
- Instrucciones de uso básicas

19.5 Commit final de limpieza:

```
git add .
git commit -m "docs: README actualizado con URL de producción y screenshots finales"
git push origin main
```

Paso 20 — Abrir tu propio Pull Request

20.1 Haz commit final del módulo de IA:

```
git add .
git commit -m "feat(ia): módulo IA completo con orquestador y clasificador de vulnerabilidades"
git push origin feature/ia
```

20.2 Abre un Pull Request de `feature/ia` a `main`. Puedes aprobarlo tú mismo ya que eres el responsable del repositorio.

Paso 21 — Documentación del módulo de IA

Escribe `/docs/manual_ia.md`:

```
# Documentación técnica – HookSuite Módulo de IA

## Stack tecnológico
- Python 3.11
- Anthropic Claude API (claude-sonnet-4-20250514)
- anthropic==0.25.0

## Arquitectura del módulo

### client.py – Cliente de la API
Encapsula todas las llamadas a la API de Anthropic.
Implementa reintentos con backoff exponencial para errores transitorios.
Parsea automáticamente respuestas JSON eliminando marcadores de código.

### prompts/ – Sistema de prompts
Un archivo por tipo de análisis. Cada prompt tiene:
- system_prompt: define el rol y el formato JSON de respuesta obligatorio
- build_user_message(): construye el mensaje con los datos del caso concreto

### analyzers/vulnerability_classifier.py – Clasificador
Orquesta las llamadas a los distintos prompts.
Filtra resultados por umbral de confianza (mínimo 60%).
Construye la ficha completa de vulnerabilidad confirmada.

### orchestrator.py – Orquestador
```

Coordina el ciclo completo: fingerprint → spider → ataques → confirmación.
Envía instrucciones a Playwright (P3) y recibe los resultados.
Confirma vulnerabilidades con múltiples ejecuciones antes de reportarlas.

Decisiones de diseño

Por qué JSON obligatorio en los prompts

Todos los prompts instruyen al modelo a responder exclusivamente en JSON.
Esto permite parsear los resultados de forma fiable sin post-procesamiento.
El campo "confianza" (0-100) permite filtrar falsos positivos.

Umbral de confianza del 60%

Se estableció tras pruebas en DVWA. Por debajo de 60% los falsos positivos superan el 30%. Por encima de 80% se pierden vulnerabilidades reales.
El equilibrio óptimo está entre 60-75% de confianza.

Confirmación de vulnerabilidades

Antes de reportar una vulnerabilidad como confirmada se ejecuta el ataque 2 veces más. Esto reduce los falsos positivos al ~5%.

Limitaciones conocidas

- Las llamadas a la API tienen latencia de 1-3 segundos por análisis
- No analiza vulnerabilidades de lógica de negocio (requiere comprensión del dominio)
- Los prompts están optimizados para SQLi y XSS; otros tipos tienen menor precisión
- El modelo puede generar falsos positivos en respuestas con mensajes de error legítimos

Estimación de costes por sesión

- Análisis de paquetes de red: ~0.002\$ por paquete
- Sesión típica de 30 minutos en DVWA: ~50 paquetes = ~0.10\$
- Análisis de resultados del Intruder (30 payloads): ~0.01\$
- Coste total estimado por sesión: ~0.15\$

Checklist final antes de la entrega

Antes del día 22, verifica que todo está en orden:

GitHub:

- ☐ El repositorio tiene las 5 ramas de módulo con commits de todos los miembros
- ☐ main tiene el historial completo con todos los módulos mergeados
- ☐ Los commits siguen la convención de CONTRIBUTING.md
- ☐ No hay credenciales ni archivos .env subidos al repositorio
- ☐ El README está actualizado con la URL de producción
- ☐ El pipeline de GitHub Actions se ejecuta sin errores
- ☐ La rama main está protegida y solo recibe merges por PR

Módulo IA:

- ☐ La API de Anthropic responde correctamente con el API Key configurado
- ☐ El prompt de análisis de paquetes detecta SQLi en DVWA
- ☐ El prompt de fingerprinting genera prioridades de ataque correctas
- ☐ El clasificador filtra por umbral de confianza del 60%
- ☐ El orquestador envía instrucciones a P3 cuando el backend está disponible
- ☐ El sistema de confirmación evita falsos positivos
- ☐ Los resultados se guardan en [results/vulnerabilities.json](#)
- ☐ El test standalone funciona sin necesitar P2 ni P3

Informe PDF:

- ☐ El informe compila con Typst sin errores
- ☐ La portada tiene nombre del proyecto, integrantes y fecha
- ☐ El índice tiene números de página correctos
- ☐ El resumen ejecutivo tiene máximo una página
- ☐ El diagrama de arquitectura técnica está incluido
- ☐ Hay evidencias (capturas) de cada módulo funcionando
- ☐ La guía de despliegue está completa y reproducible
- ☐ El manual de uso cubre todas las funcionalidades
- ☐ Las conclusiones son reflexivas y concretas
- ☐ El road map de P4 está integrado correctamente
- ☐ El PDF se sube al campus virtual antes del día 25

Documentación:

- ☐ [/docs/manual_ia.md](#) está completo
- ☐ Las capturas del módulo IA están en [/docs/capturas/ia/](#)
- ☐ El informe final está en [/docs/informe/informe_hooksuite.pdf](#)

Capturas para el informe PDF

Guarda en [/docs/capturas/ia/](#):

- [api_funcionando.png](#) — Salida del test de la API mostrando respuesta de Claude
- [analisis_sql_i.png](#) — Resultado del clasificador detectando una vulnerabilidad SQLi
- [fingerprint_prioridades.png](#) — Análisis de fingerprinting con prioridades de ataque
- [vulnerabilidades_json.png](#) — El archivo [vulnerabilities.json](#) con hallazgos reales
- [test_standalone_output.png](#) — Salida completa del test standalone
- [github_commits.png](#) — Vista del historial de commits en GitHub mostrando todos los roles
- [github_actions.png](#) — Pipeline de GitHub Actions ejecutándose correctamente
- [pull_requests.png](#) — Vista de Pull Requests mergeados de todos los módulos